Personalized Grocery and Diet Planning Application (NutriSpoke)

Course: CP317

Group Number: 3

Names:

Zulqarnain Ali
Fatima Afzal
Dezirae Asante
Toshani Settivari
Bassil Obeidi

Roles:

Q/A & Testing: Toshani Settivari

Product Owner(s): **Dezirae Asante, Fatima Afzal**

Software Engineers: Bassil Obeidi, Dezirae Asante

UI/UX Designers: Zulqarnain Ali, Fatima Afzal

Scrum Master*:* Bassil Obeidi

Table Of Contents:

1. Sprint Goal Page 3
2. Sprint Backlog Page 3-4
3. Implementation Progress Page 5-7
4. Testing and Review Page 7-8
5. Team Blog Page 8
6. Appendix Page 9

1. Sprint Goal

The primary objective of Sprint 1 is to establish the foundational functionality of the application through the development and integration of three core modules: user onboarding, personalization, and nutritional estimation. The onboarding module (PROF-01) will collect basic user information, while the personalization module (PROF-02) will allow users to specify their dietary goals, preferences, and restrictions. Both modules will also allow users to skip optional questions, ensuring that the application remains functional even when certain information is not provided. This information will be stored in a shared data store and retrieved by the nutritional estimation module (PROF-03), which will calculate personalized daily caloric and macronutrient targets using established formulas. Where user information is unavailable, the system will use default values to generate estimates. These modules will be integrated into a functional web application, allowing user information to be collected, stored, and used across the system while providing a consistent experience as users navigate between features.

These backlog items were prioritized as all subsequent features rely on the information obtained from these backlog items. In order to test the subsequent items, these must be implemented first. Additionally, the main offering of our software is personalization, and as such these are the highest priority.

2. Sprint Backlog

ID	PROF-01
Title	User Onboarding and Contextualization
Points	3
Priority	High
Assigned Members	Bassil, Zulqarnain
Tasks	Intro page. Onboarding page. Basic information inputs. Navigation button (skip button, submit buttons).

	Convert units (to metric). Validate inputs. Json storage on submit.
Status	Complete

ID	PROF-02
Title	Preferences & Restrictions
Points	3
Priority	High
Assigned Members	Bassil, Zulqarnain
Tasks	Preferences page. More customization options - Goals, Allergies, Diets etc. Navigation button (skip button, submit buttons). Connect to existing pages. Json storage on submit.
Status	Complete

ID	PROF-03
Title	Nutritional Targets
Points	5
Priority	High
Assigned Members	Bassil, Zulqarnain
Tasks	Nutritional targets page. Retrieve information from the previously captured user info. Use Mifflin-St Jeor equation to calculate maintenance calorie recommendation. Display recommended Macro split over the recommended caloric recommendation using a predefined algorithm/formula. Use captured user info (if any) to personalize these targets. Create an alternative method to calculate recommendations when info is missing.
Status	Complete

3. Implementation Progress

Features Implemented

During this sprint, our team developed a working NutriSpoke web prototype using Reflex, a python library for creating web-applications. The application connects three main components: user onboarding, dietary preferences, and nutritional target generation. Users can enter their information, customize their preferences, and view their daily nutritional targets through a connected workflow.

The front end was designed to be clean, minimalistic and provide an intuitive user experience. Each input field includes a brief explanation of its purpose and any applicable restrictions to help users enter information correctly. Additionally, the nutritional targets and home page include a disclaimer clarifying that recommendations are estimates.

The onboarding page collects sex, age, height, weight, and activity level. Height and weight can be entered in metric or imperial units, with imperial measurements converted to metric before being stored. Users can skip optional fields and receive nutrition estimates based on default values. If they enter an invalid value, the application displays an error message and requires them to correct it before continuing.

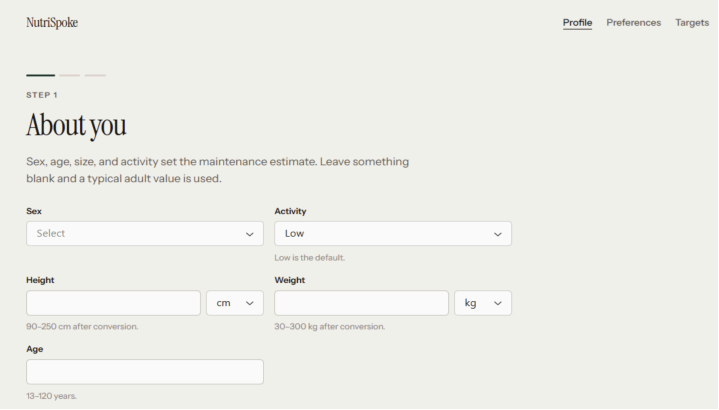


Figure 1: Profile page for entering personal information.

On the preferences page, users can select their weight goal, dietary preferences, and allergies. They can also set calorie and fibre targets and choose the optional Muscle Gain setting. These preferences help personalize nutritional targets and will guide future meal and grocery recommendations. Several fields use predefined choices to simplify the form and make inputs easier to validate.

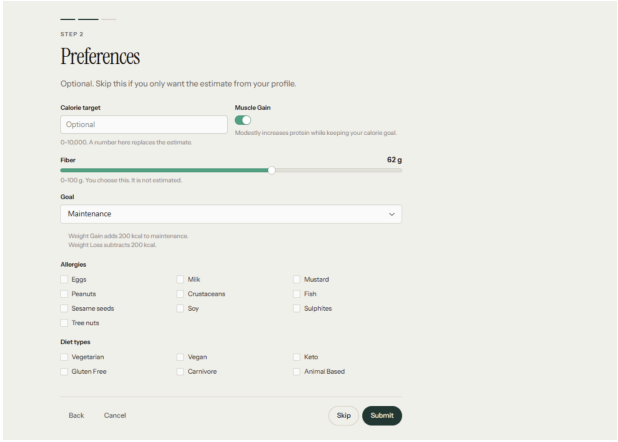


Figure 2: Preferences page for dietary goals and restrictions.

Profile and preference information is saved in two JSON files. The nutritional targets module retrieves this information to calculate the user's daily targets. JSON storage was sufficient for the small amount of information needed by the current prototype. A screenshot of the Homepage is provided in the appendix.

Nutritional Target Generation

The application uses the Mifflin–St Jeor equation to calculate basal metabolic rate, then applies an activity multiplier to estimate maintenance calories. If the user selects Weight Gain or Weight Loss, the application adjusts the maintenance estimate by adding or subtracting 200 calories. A valid manually entered calorie target overrides the calculated calorie recommendation and these goal adjustments.

The application also calculates daily protein, carbohydrate, and fat targets. Before the Muscle Gain adjustment, protein is set to the smaller of one gram per kilogram of body weight or the gram equivalent of 25% of daily calories. Fat provides 30% of calories, while carbohydrates account for the remaining calories. The optional Muscle Gain setting increases the percentage of daily calories from protein. Users can also set their daily fibre target, which defaults to 30 grams if no value is entered.

The application uses default values for any profile fields left blank and includes the information the user has entered when calculating nutrition estimates.

The results page displays daily calorie, protein, carbohydrate, fat, and fibre targets. Numerical values and a segmented macronutrient bar help users interpret the results.

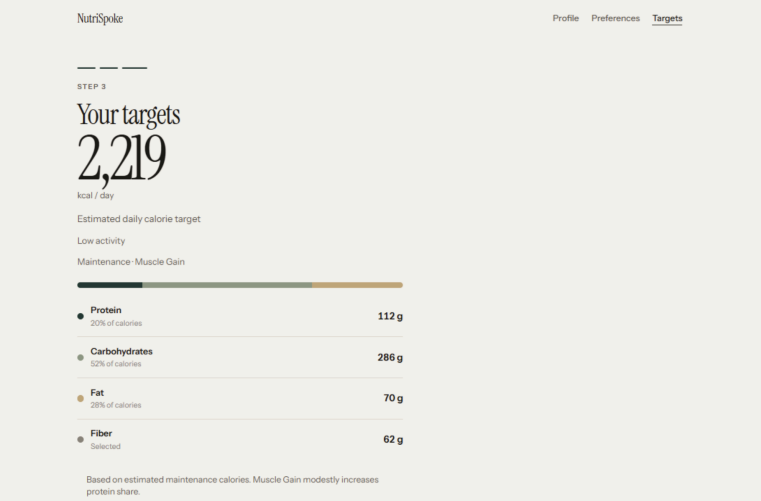


Figure 3: Results page that displays the estimated daily calorie and nutrient intake recommendations.

Testing and Technical Challenges

The implemented features were checked through manual front-end testing and 31 automated back-end unit tests. These tests covered profile generation, navigation, unit conversions, input validation, saved preferences, default values, and nutritional calculations.

One issue identified during initial testing was that invalid inputs prevented users from continuing without displaying an explanatory message. The developers added the missing notification, and the application was tested again. All front-end and back-end tests passed in the second round, and the Product Owner verified the three profile-related stories as Done.

Another challenge was supporting incomplete information without causing calculation errors. Missing fields are handled independently through fallback values. Missing, unreadable, or malformed JSON files are treated as empty records, allowing the calculation module to use defaults instead of failing.

Work Deferred to Future Sprints

Open-ended preference inputs were deferred to a later sprint. The current prototype uses fixed choices for activity levels, goals, allergies, and diet types to limit the complexity of validating user input.

The prototype currently stores data in JSON files. As more features are added, the team may introduce a database such as SQLite to manage the additional data.

4. Testing and Review

Our team verified the functionality of NutriSpoke through manual front-end testing and back-end unit testing. Manual tests were used to simulate user interactions, including entering profile information, skipping optional information, navigating between pages, using unit conversions and more. Acceptance criteria were checked by comparing the expected results of each user story with the actual behaviour of the application. A detailed list of all individual front-end and back-end tests can be found in the testing spreadsheet attached to this submission.

The manual testing showed that profile generation, base estimates, and unit conversions met their acceptance criteria. Input validation successfully prevented invalid data types and prevented users from continuing with invalid values; however, the application did not display an “invalid input” error message. Navigation testing showed that the NutriSpoke logo correctly returns users to the home page, and the Profile, Preferences, and Targets navigation behaviour matched their acceptance criteria.

Back-end functionality was also verified using 31 unit tests across test_backend.py. These tests covered metric and imperial conversions, boundary and invalid input handling, blank submissions, calorie and fibre validation, saved preferences, activity levels, default values,

calorie and macronutrient calculations, goals, manual calorie overrides, and page registration. All listed back-end unit tests passed upon the first test run.

After the first round of testing was complete, developers were informed of the failed front-end (missing notification upon invalid input) test. After developers implemented the required functional requirement, testing was run once again. All front-end and back-end tests passed on the second round of testing.

Overall, the core nutrition calculation data validation, and associated acceptance criteria for PROF-01 to 03 were successfully verified.

Testing Summary:

Story	Status Verified by Product Owner(s)	Method of Checking Acceptance Criteria	Notes
PROF-01	Pass. Done.	Unit testing for data and input handling, missing values do not impact program functionality. Manual testing for GUI functionality and saved data usage.	Required Fix and Second round of testing to verify 1 failed front-end test case.
PROF-02	Pass. Done.	Unit testing for data and input handling, missing values do not impact program functionality. Manual testing for GUI functionality and saved data usage.	All tests passed upon the first and second round of testing.
PROF-03	Pass. Done.	Unit testing for nutritional recommendation calculations, dynamic targets and robustness. Manual testing for GUI functionality and saved data usage.	All tests passed upon the first and second round of testing.

A detailed testing spreadsheet is attached to this submission titled “Sprint_1_Testing.xlsx”.

5. Updated Team Blog

Present in an excel sheet attached to this submission.

6. Appendix

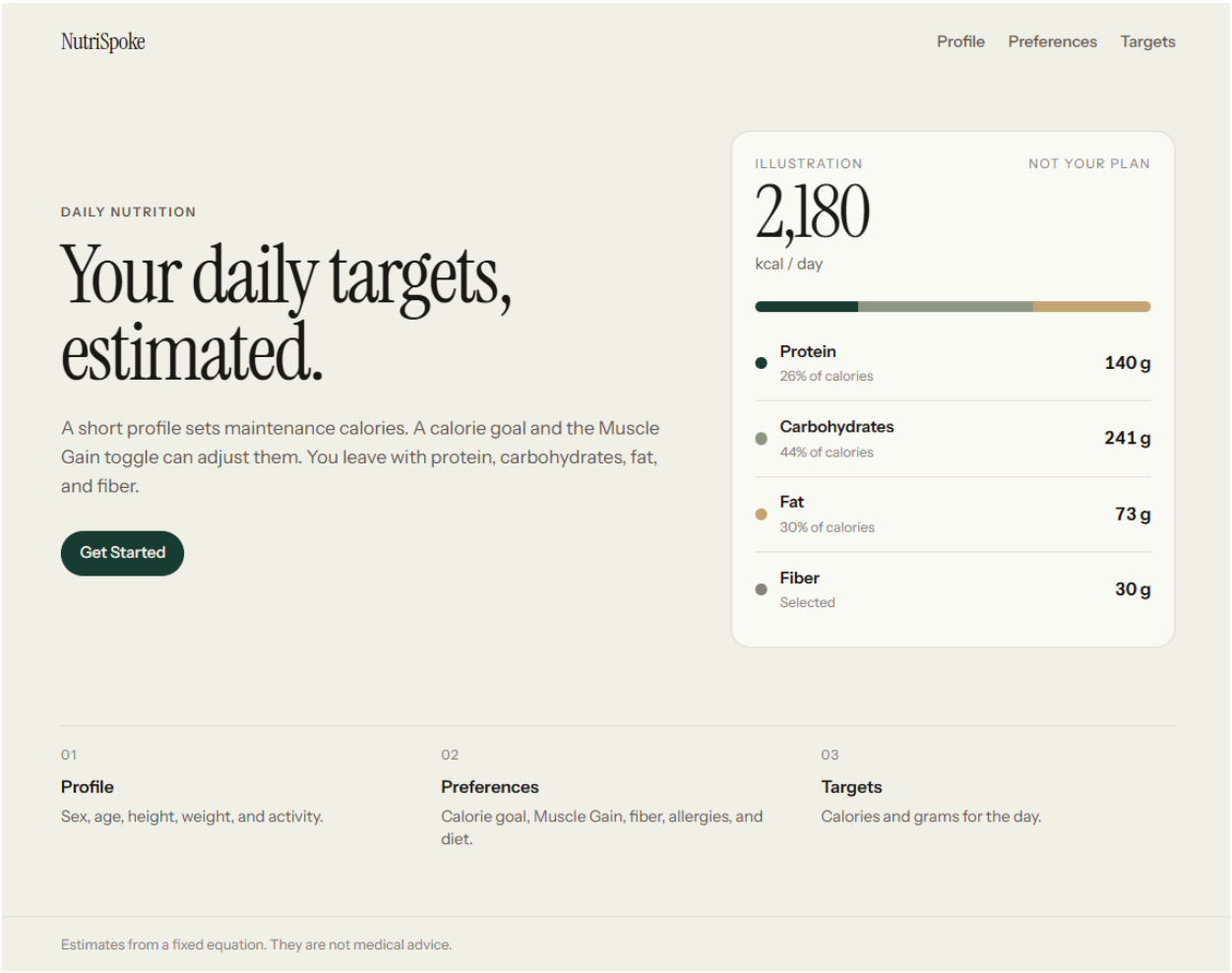


Figure 4: NutriSpoke Homepage.